



Ikaskuntza Birtual eta Digitalizatuen LHII
CIFP de Aprendizajes Virtuales y Digitalizados

DIW04 Sass: Syntactically Awesome Style Sheets

Izaro Arbaiza Ipas
DAW - DIW

Curso: 2025/26

EUSKO JAURLARITZA



GOBIERNO VASCO

HEZKUNTZA SAILA
Lanbide Heziketako Sailburuordetza

DEPARTAMENTO DE EDUCACIÓN
Viceconsejería de Formación Profesional

Índice

1. INTRODUCCIÓN	1
2. DIFERENTES APARTADOS DE LA TAREA	1
2.1. Variables	1
2.2. Operador aritmético	1
2.3. Reglas anidadas	2
2.4. @use	2
2.5. @media	3
2.6. @extend	3
2.7. @if	4
2.8. @mixin	4
2.9. Fichero comprimido	4

1. INTRODUCCIÓN

En esta tarea se ha realizado la migración del diseño web previo a una arquitectura basada en Sass (SCSS). El objetivo principal ha sido modularizar el código y añadir lógica de programación para que el mantenimiento de la hoja de estilos sea más eficiente y profesional.

2. DIFERENTES APARTADOS DE LA TAREA

2.1. Variables

Se han sustituido valores repetidos (colores, tipografías, sombras, etc.) por variables definidas en el archivo `_variables.scss`.

Variables (Antes de la modificación) En el CSS original se utilizaban valores directos como: <pre>background-color: #F8F6F6; color: #111827;</pre>	Variables (Modificación realizada) Declaración en <code>_variables.scss</code> <pre>css > _variables.scss > [a] \$shadow-s 1 \$body-bg: #F8F6F6; 2 \$text-color: #111827;</pre> Uso dado a la variable en <code>_layout.scss</code> <pre>body { line-height: 1.5; -webkit-font-smoothing: anti background-color: \$body-bg; color: \$text-color;</pre>
--	---

2.2. Operador aritmético

Se ha utilizado un operador aritmético para calcular valores dinámicamente.

Operador aritmético (Antes de la modificación) El padding se definía manualmente: <pre>.header-container { padding: 18px 17.5%; display: flex;</pre>	Operador aritmético (Modificación realizada) <pre>\$layout-vertical-padding: 10px + 8px;header-container { padding: \$layout-vertical-padding 17.5%; display: flex;</pre>
--	--



2.3. Reglas anidadas

Se han anidado selectores para reflejar la estructura del HTML y evitar repetición.

Reglas anidadas (Antes de la modificación)	Reglas anidadas (Modificación realizada)
<pre><code>.nav-actions .lang-selector .lang-option:hover { color: #E64C19; }</code></pre>	<pre><code>.nav-actions { @include flex-center(row, space-between, center); gap: 25px; .icon-btn { ... } .lang-selector { display: flex; background-color: \$white; border-radius: 6px; border: 1px solid #d1d5db; font-size: 0.8rem; .lang-option { padding: 5px 10px; cursor: pointer; &:hover { color: \$brand-color; } } } }</code></pre>

2.4. @use

Se ha utilizado la directiva @use para modularizar el proyecto en distintos archivos SCSS, permitiendo separar la lógica de estilos en diferentes módulos reutilizables.

@import (Antes de la modificación)	@import (Modificación realizada)
En el CSS original no existía ningún sistema de importación o modularización, ya que todo el código estaba en un único archivo CSS.	En el archivo principal style.scss se han importado los distintos módulos creados: <pre><code>1 @use "variables" as *; 2 @use "mixins" as *; 3 @use "layout"; 4 @use "components"; 5</code></pre>



2.5. @media

Se han mantenido las reglas responsive, pero integrándolas dentro de la estructura SCSS, mejorando su organización junto al resto de estilos.

@media (Antes de la modificación)

No existían reglas @media en el CSS original; todo estaba pensado para pantallas grandes, por lo que el diseño no se adaptaba a móviles o tablets.

@media (Modificación realizada)

```
@media (max-width: 768px) {
  .header-container {
    flex-direction: column;
    align-items: center;
  }
  .cat-container {
    padding: 10px;
  }
}
```

2.6. @extend

Se ha creado un placeholder reutilizable para estilos comunes.

@extend (Antes de la modificación)

Se repetían propiedades en varios botones.

@extend (Modificación realizada)

```
.nav-actions {
  @include flex-center(row, space-between, center);
  gap: 25px;
  .icon-btn {
    @extend %base-btn;
    font-size: 1.2rem;
    color: #6b7280;
    &:hover {
      color: $brand-color;
    }
  }
}
```



2.7. @if

Se ha utilizado lógica condicional para aplicar estilos dinámicos.

@if (Antes de la modificación) No existía lógica condicional en CSS.	@if (Modificación realizada) <pre>@if \$use-dark-theme { footer { background-color: color.adjust(\$footer-bg, \$lightness: -5%); } }</pre>
---	--

2.8. @mixin

Se han creado mixins para reutilizar bloques de código.

@mixin (Antes de la modificación) Se repetían propiedades como display: flex, justify-content, etc.	@mixin (Modificación realizada) <pre>@mixin flex-center(\$direction: row, \$justify: center, \$align: center) { display: flex; flex-direction: \$direction; justify-content: \$justify; align-items: \$align; }</pre> Uso <pre>.nav-actions { @include flex-center(row, space-between, center); }</pre>
--	--

2.9. Fichero comprimido

Explicación cambios realizados. En las capturas deben indicarse el número de las líneas del CSS y del SCSS antes de comprimirse.

Fichero sin comprimir (Primeras 10 líneas) <pre>1 @charset "UTF-8"; 2 .nav-actions .icon-btn { 3 display: inline-flex; 4 align-items: center; 5 border-radius: 8px; 6 } 7 8 *, *::before, *::after { 9 margin: 0; 10 padding: 0;</pre>	Fichero modificado <pre>1 {"version":3,"sources":["style.css","_layout.scss","_mixins.scss","_variables.scss","_components.scss"],</pre>
--	---

